

Skill - 24:

```
sahasrasagi@Sahasra: ~/OS_L  ×  +  ▾
GNU nano 8.7.1 skill_24.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <signal.h>
#include <sys/wait.h>

#define MAX_ARGS 32
#define MAX_JOBS 16

typedef struct {
    int id;
    pid_t pid;
    int active;
} Job;

static Job jobs[MAX_JOBS];
static int next_job = 1;
static volatile sig_atomic_t fg_pid = -1;

static void sigint_handler(int sig) {
    (void)sig;

    if (fg_pid > 0)
        kill(fg_pid, SIGINT);
    else
        write(STDOUT_FILENO, "\nUse 'exit' to quit.\n", 21);
}

static void add_job(pid_t pid) {
    for (int i = 0; i < MAX_JOBS; i++) {
        if (!jobs[i].active) {
            jobs[i] = (Job){next_job++, pid, 1};
            printf("[%d] %d\n", jobs[i].id, pid);
            return;
        }
    }
    printf("Job table full.\n");
}
```

```
sahasrasagi@Sahasra: ~/OS_1 × + v
GNU nano 8.7.1 skill_24.c
static void update_jobs(void) {
    for (int i = 0; i < MAX_JOBS; i++) {
        if (!jobs[i].active)
            continue;

        int status;
        pid_t r = waitpid(jobs[i].pid, &status, WNOHANG);

        if (r == jobs[i].pid) {
            printf("[%d] Done\n", jobs[i].id);
            jobs[i].active = 0;
        }
    }
}

static int tokenize(char *line, char **args) {
    int n = 0;
    char *tok = strtok(line, " \t");

    while (tok && n < MAX_ARGS - 1) {
        args[n++] = tok;
        tok = strtok(NULL, " \t");
    }

    args[n] = NULL;
    return n;
}

static void run_simple(char **args, int background) {
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        signal(SIGINT, SIG_DFL);
        execvp(args[0], args);
        perror("execvp");
    }
}
```

```
static void run_pipeline(char *line) {
    char *parts[8];
    int count = 0;

    char *p = strtok(line, "|");
    while (p && count < 8) {
        parts[count++] = p;
        p = strtok(NULL, "|");
    }

    if (count < 2) {
        printf("Invalid pipeline.\n");
        return;
    }

    int pipes[7][2];
    for (int i = 0; i < count - 1; i++)
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
            return;
        }

    pid_t pids[8];

    for (int i = 0; i < count; i++) {
        char *args[MAX_ARGS];
        int argc = tokenize(parts[i], args);

        if (argc == 0)
            continue;

        pids[i] = fork();

        if (pids[i] == -1) {
            perror("fork");
            return;
        }

        if (pids[i] == 0) {
            signal(SIGINT, SIG_DFL);
```

```

int main(void) {
    char line[1024];

    struct sigaction sa;
    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, NULL);

    printf("=====\n");
    printf(" OOSP FINAL DEMO - MINI UNIX SHELL\n");
    printf("=====\n");
    printf("Supported: normal commands, |, &, jobs, cd, pwd, exit\n\n");

    while (1) {
        update_jobs();

        printf("oosp> ");
        fflush(stdout);

        if (!fgets(line, sizeof(line), stdin))
            break;

        line[strcspn(line, "\n")] = '\0';

        if (line[0] == '\0')
            continue;

        if (strcmp(line, "exit") == 0)
            break;

        if (strcmp(line, "pwd") == 0) {
            char cwd[1024];
            if (getcwd(cwd, sizeof(cwd)))
                printf("%s\n", cwd);
            else
                perror("getcwd");
            continue;
        }

        if (strncmp(line, "cd ", 3) == 0) {
            if (chdir(line + 3) == -1)
                perror("cd");
            continue;
        }

        if (strcmp(line, "jobs") == 0) {
            for (int i = 0; i < MAX_JOBS; i++)
                if (jobs[i].active)
                    printf("[%d] Running pid=%d\n",
                        jobs[i].id, jobs[i].pid);
            continue;
        }

        if (strchr(line, '|')) {
            run_pipeline(line);
            continue;
        }
    }
}

```

```

        if (strcmp(args[argc - 1], "&") == 0) {
            background = 1;
            args[--argc] = NULL;
        }

        run_simple(args, background);
    }

    // Reap remaining background jobs.
    for (int i = 0; i < MAX_JOBS; i++)
        if (jobs[i].active)
            waitpid(jobs[i].pid, NULL, 0);

    printf("Final demo ended cleanly.\n");
    return 0;
}

```

```
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_24
=====
OSSP FINAL DEMO - MINI UNIX SHELL
=====
Supported: normal commands, |, &, jobs, cd, pwd, exit

ossp> ls
a.out      program2.c      skill_10.c      skill_13      skill_15.c      skill_18      ski
output.txt shell_errors.log skill_11      skill_13.c      skill_16      skill_18.c      ski
program1   shellforge      skill_11.c      skill_14      skill_16.c      skill_19      ski
program1.c shellforge.c     skill_12      skill_14.c      skill_17      skill_19.c      ski
program2   skill_10        skill_12.c      skill_15      skill_17.c      skill_20      ski
ossp> pwd
/home/sahasrasagi/OS_Lab/skill
ossp> date
Tue Oct 6 14:21:26 UTC 2026
ossp> cal
      October 2026
Su Mo Tu We Th Fr Sa
                1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31

ossp> exit
Final demo ended cleanly.
sahasrasagi@Sahasra:~/OS_Lab/skill$ |
```